

# Program Anomaly Detection With 1D CNNs

Anonymous Authors

*Abstract*—**TODO**

## I. INTRODUCTION

The globalization of semiconductor manufacturing has separated chip design from chip fabrication. A design house specifies a circuit and hands it to an external foundry to manufacture, often alongside third-party intellectual property blocks and CAD tooling supplied by still other parties [1]. Each of these external actors, the IP vendor, the CAD tool provider, the foundry itself, has the technical means to alter the design before it becomes silicon, and none of them is fully verifiable by the party that ultimately deploys the chip. A hardware Trojan is a malicious modification introduced at any of these stages: a deliberate alteration to a circuit’s logic that leaves its specified behavior unchanged under normal operation, but that activates a hidden payload, leaking secret data, corrupting computation, degrading reliability, or granting covert access, when a specific, usually rare, trigger condition occurs [1], [2]. Because the modification is engineered to remain functionally invisible until that trigger fires, a Trojan-infected chip passes standard functional verification and manufacturing tests in the same way a genuine chip would [3].

Two decades of research have produced an extensive body of work on inserting, detecting, and defending against hardware Trojans under this untrusted-foundry model [4], driven by the recognition that a single point of malicious insertion at the design or fabrication stage can compromise every unit that stage subsequently produces. What makes this threat class distinct from conventional hardware faults is that it is adversarial by design: a rare-condition trigger is deliberately engineered to survive whatever pre-deployment test campaign the defender runs, since the attacker who inserts it controls exactly how rare and how well-disguised that condition needs to be to evade detection [1], [5]. Firmware signing does not close this gap. Signing attests that the software running on a chip is unmodified; it says nothing about whether the silicon executing that software was fabricated as specified. A Trojan inserted at the RTL or fabrication stage, with a trigger chosen specifically to remain dormant through functional verification and manufacturing test, survives signing intact, because signing has nothing to say about the substrate the signed firmware runs on.

Countermeasures against hardware Trojans have accordingly been developed across the entire chip lifecycle [1]. Pre-silicon methods examine the design itself, before it is committed to silicon at all, through statistical test-pattern generation targeting a Trojan’s rare trigger nets [5] or gate-level information-flow tracking on the synthesized netlist [6]. Post-silicon methods examine the fabricated chip, but only

within a controlled test environment, before or apart from field deployment, comparing power or timing signatures against a golden reference [7]. Both share the same structural blind spot against the threat this work targets: each depends on exercising the design during a bounded window, and a trigger deliberately engineered to be rare is, almost by definition, unlikely to fire inside that window [2]. Runtime monitoring is the remaining option, and the only one that observes a chip continuously during its actual operation, giving a rare trigger its sole genuine opportunity to activate and be caught [3].

Current runtime detection work spans several distinct observable planes. Hardware performance counters and on-chip statistical monitors periodically sample aggregate behavioral metrics, such as instruction throughput or cache activity, and flag deviations from an expected profile [8], [9]; because they track statistics rather than individual transaction content, a payload engineered to leave those aggregates unchanged passes undetected, and several instances additionally assume a trusted baseline that is unavailable when the observed design is itself untrusted [3]. Physical side-channel methods instrument a chip’s electromagnetic or power emissions instead, moving observation off the digital datapath entirely at the cost of a signal-to-noise problem that worsens at smaller process nodes [2]. Other approaches observe on-chip activity more directly, but typically check it against a specification decided in advance rather than the raw transactions themselves, which limits how well they generalize beyond the specific design they were built for [10].

[TODO: our work presents para]

## II. RELATED WORK

Prior runtime hardware Trojan detection methodologies observe different signals to capture processor or system behavior and flag anomalies, ranging from aggregate performance statistics to physical emissions to the content of on-chip communication itself. We group this work into five families based on the observable each relies on.

### A. Hardware Performance Counter Monitoring

Detecting anomalies using traces of microarchitectural events collected by Hardware Performance Counters (HPCs) has garnered significant interest as a hardware-assisted alternative to traditional software-based malware detection. HPCs are specialized processor registers that record low-level hardware events such as cache misses, branch mispredictions, retired instructions, and CPU cycles [11]. These event traces are typically used as input features for machine learning classifiers to discern malicious software from benign applications [12]. Although HPC-based malware detectors have demonstrated high

accuracy on known malware families, their performance often degrades when generalizing to previously unseen malware or diverse workloads, requiring more complex techniques for accurate classification [13]. Furthermore, modern processors provide only a limited number of HPC registers, restricting the number of hardware events that can be monitored simultaneously and consequently limiting the amount of information available for accurate classification [11].

### B. EM/Power Side-Channel Runtime Sensing

A separate line of runtime detection looks at the chip's physical emissions, using electromagnetic radiation or power consumption as a proxy for internal switching activity. Recent instances of this approach are golden-model-free: detection and localization are performed by sensing anomalous EM or power activity during live operation through on-chip sensor arrays [14], or by applying deep-learning pipelines to runtime scans, without requiring a trusted reference device [15]. The core limitation is signal-to-noise degradation under process variation, which worsens at smaller technology nodes and remains the primary obstacle for side-channel-based detection generally, regardless of whether a golden reference is required.

### C. Statistical Communication-Channel Monitoring

Communication traffic between on-chip modules offers a third observable, distinct from performance counters and physical emissions. One approach in this space statistically models the communication channel's traffic patterns and checks them against assertions embedded in the RTL at design time [10]. This avoids needing a golden reference chip, but has two limits: the assertions are hand-written per SoC, and detection only catches shifts in aggregate traffic statistics, not anomalies in the content of individual transactions.

### D. Control-Flow Attestation

Control-flow attestation (CFA) observes execution paths rather than aggregate statistics or physical emissions. It verifies whether the sequence of blocks executed matches its expected control flow graph. This is done using a trusted execution environment (TEE), which produces a signed record of the observed path [16]. Earlier hardware-based schemes computed a cumulative hash of the code blocks to reduce computational overhead. However, this only worked for small programs because, if a program had loops, it would cause the hash to blow up exponentially [17]. A recent development proposes a solution to the exponential problem by keeping track of which blocks have executed and verifying the path's consistency with forward and backward checks. This leads to linear growth instead of an exponential one and results in quicker and cheaper computation [18]. CFA relies on a TEE rather than a golden reference chip. However, if a Trojan does not change the sequence of the code blocks, but alters and corrupts data on the expected normal path, it would remain undetected.

### E. Software-Level Defenses

Software-based approaches have also been proposed to address hardware Trojans without modifying existing hardware. DETON [19] employs code obfuscation techniques to mitigate information leakage from compromised processors by reducing the exposure of sensitive data during execution. Other works generate multiple functionally equivalent program variants to diversify execution patterns, enabling the detection of Trojan-induced inconsistencies or avoiding specific trigger conditions [20]. However, these approaches introduce additional execution overhead and primarily rely on software transformations rather than directly monitoring hardware behavior.

## III. PROPOSED METHOD

## IV. EXPERIMENTAL RESULTS

## V. DISCUSSION

## VI. CONCLUSIONS AND FUTURE WORK

## REFERENCES

- [1] M. Tehranipoor and F. Koushanfar, "A survey of Hardware Trojan taxonomy and detection," *IEEE Design & Test of Computers*, vol. 27, no. 1, pp. 10–25, 2010.
- [2] G. Di Natale and E. Valea, "Hardware Trojans Horses: Two decades later, what we really know," in *2026 31st IEEE European Test Symposium (ETS)*. IEEE, 2026.
- [3] B. Amornpaisannon, A. Diavastos, L.-S. Peh, and T. E. Carlson, "Secure run-time Hardware Trojan detection using lightweight analytical models," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 2, pp. 431–441, 2024.
- [4] S. Bhunia and M. Tehranipoor, *The Hardware Trojan War: Attacks, Myths, and Defenses*. Cham, Switzerland: Springer, 2018.
- [5] R. S. Chakraborty, F. Wolff, S. Paul, C. Papachristou, and S. Bhunia, "MERO: A statistical approach for Hardware Trojan detection," in *Cryptographic Hardware and Embedded Systems (CHES)*, ser. LNCS, vol. 5747. Springer, 2009, pp. 396–410.
- [6] W. Hu, B. Mao, J. Oberg, and R. Kastner, "Detecting Hardware Trojans with gate-level information-flow tracking," *Computer*, vol. 49, no. 8, pp. 44–52, 2016.
- [7] D. Agrawal, S. Baktir, D. Karakoyunlu, P. Rohatgi, and B. Sunar, "Trojan detection using IC fingerprinting," in *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2007, pp. 296–310.
- [8] R. Elnaggar, K. Chakrabarty, and M. B. Tahoori, "Run-time Hardware Trojan detection using performance counters," in *Proceedings of the IEEE International Test Conference (ITC)*. IEEE, 2017, pp. 1–10.
- [9] —, "Hardware Trojan detection using changepoint-based anomaly detection techniques," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 27, no. 12, pp. 2706–2719, 2019.
- [10] F. Khalid, S. R. Hasan, S. Zia, O. Hasan, F. Awwad, and M. Shafique, "SIMCom: Statistical sniffing of inter-module communications for run-time Hardware Trojan detection," *Journal of Systems Architecture*, 2019, also arXiv:1901.07299.
- [11] C. P. Chenet, A. Savino, and S. Di Carlo, "A survey on hardware-based malware detection approaches," *IEEE Access*, vol. 12, pp. 54 115–54 128, 2024.
- [12] Z. He, A. Rezaei, H. Homayoun, and H. Sayadi, "Deep neural network and transfer learning for accurate hardware-based zero-day malware detection," in *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22)*. ACM, 2022, pp. 27–32.
- [13] J. Pattee, S. M. Anik, and B. K. Lee, "Performance monitoring counter based intelligent malware detection and design alternatives," *IEEE Access*, vol. 10, pp. 28 685–28 692, 2022.
- [14] H. Wang, M. Panoff, Z. Zhan, S. Wang, C. Bobda, and D. Forte, "Programmable EM sensor array for golden-model free run-time Trojan detection and localization," in *Design, Automation and Test in Europe Conference (DATE)*, 2024.

- [15] H. Wang, M. Panoff, S. Wang, and D. Forte, "HT-EMIS: A deep learning tool for Hardware Trojan detection and identification through runtime EM side-channels," in *Proceedings of the Great Lakes Symposium on VLSI (GLSVLSI '23)*. ACM, 2023, pp. 51–56.
- [16] T. Abera, N. Asokan, L. Davi, J.-E. Ekberg, T. Nyman, A. Paverd, A.-R. Sadeghi, and G. Tsudik, "C-FLAT: Control-flow attestation for embedded systems software," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 743–754.
- [17] G. Dessouky, S. Zeitouni, T. Nyman, A. Paverd, L. Davi, P. Koeberl, N. Asokan, and A.-R. Sadeghi, "LO-FAT: Low-overhead control flow attestation in hardware," in *Proceedings of the 54th Annual Design Automation Conference (DAC)*. ACM, 2017.
- [18] M. Armanuzzaman, E. Kirda, and Z. Zhao, "ENOLA: Efficient control-flow attestation for embedded systems," *arXiv preprint arXiv:2501.11207*, 2025.
- [19] L. Cassano, M. Iamundo, T. A. Lopez, A. Nazzari, and G. Di Natale, "DETON: DEfeating Hardware Trojan horses in microprocessors through software Obfuscation," *Journal of Systems Architecture*, vol. 129, p. 102592, 2022.
- [20] M. Hasan, J. Cruz, P. Chakraborty, S. Bhunia, and T. Hoque, "Software variants for Hardware Trojan detection and resilience in COTS processors," *arXiv preprint arXiv:2112.00304*, 2021.